

CSE445: Real-Time Phishing URL Detection Using Machine Learning

This notebook implements the project proposal using the **LegitPhish Dataset**.

Class mapping

- 0 = Phishing
- 1 = Legitimate

Six models

1. Logistic Regression
2. SVM
3. Random Forest
4. XGBoost
5. K-Means Clustering
6. LLM-based tabular classifier

1. Install required packages

Run this cell in Google Colab.

```
!pip -q install pandas numpy scikit-learn matplotlib joblib xgboost transformer
```

2. Upload the LegitPhish CSV

Upload `url_features_extracted1.csv`.

```
from google.colab import files
uploaded = files.upload()
```

`Choose Files` url_features...xtracted.csv
url_features_extracted.csv(text/csv) - 8963838 bytes, last modified: 8/26/2026 - 100% done
Saving url_features_extracted.csv to url_features_extracted (1).csv

3. Imports and configuration

```
import io, os, re, math, json, warnings
from collections import Counter
from urllib.parse import urlparse, parse_qs

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.cluster import KMeans
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, classification_report, confusion_matrix,
    ConfusionMatrixDisplay, RocCurveDisplay
)
from xgboost import XGBClassifier

warnings.filterwarnings("ignore")
RANDOM_STATE = 42
```

4. Load and inspect dataset

```
filename = next(iter(uploaded.keys()))
df = pd.read_csv(io.BytesIO(uploaded[filename]))
df.columns = [str(c).strip() for c in df.columns]

print("Shape:", df.shape)
display(df.head())
print(df.info())
print("\nClass distribution:")
print(df["ClassLabel"].value_counts().sort_index())
```

Shape: (101219, 18)

	URL	url_length	has_ip_address	dot_cou
0	https://keraekken-loagginnusa.godaddysites.com/	47	0	
1	https://metamsk01lgiix.godaddysites.com/	40	0	
2	http://myglobaltech.in/	23	0	
3	http://djtool-for-spotify.com/	30	0	
4	https://scearmcoommunnltty.com/invent/freind/get	47	0	

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 101219 entries, 0 to 101218

Data columns (total 18 columns):

#	Column	Non-Null Count	Dtype
0	URL	101219 non-null	object
1	url_length	101219 non-null	int64
2	has_ip_address	101219 non-null	int64
3	dot_count	101219 non-null	int64
4	https_flag	101219 non-null	int64
5	url_entropy	101219 non-null	float64
6	token_count	101219 non-null	int64
7	subdomain_count	101219 non-null	int64
8	query_param_count	101219 non-null	int64
9	tld_length	101219 non-null	int64
10	path_length	101219 non-null	int64
11	has_hyphen_in_domain	101219 non-null	int64
12	number_of_digits	101219 non-null	int64
13	tld_popularity	101219 non-null	int64
14	suspicious_file_extension	101219 non-null	int64
15	domain_name_length	101219 non-null	int64
16	percentage_numeric_chars	101219 non-null	float64
17	ClassLabel	101218 non-null	float64

dtypes: float64(3), int64(14), object(1)

memory usage: 13.9+ MB

None

Class distribution:

ClassLabel

0.0 63678

1.0 37540

Name: count, dtype: int64

✓ 5. Cleaning and feature selection

```
FEATURES = [
    "url_length", "has_ip_address", "dot_count", "https_flag",
    "url_entropy", "token_count", "subdomain_count", "query_param_count",
```

```

    "tld_length", "path_length", "has_hyphen_in_domain", "number_of_digits",
    "tld_popularity", "suspicious_file_extension",
    "domain_name_length", "percentage_numeric_chars"
]

TARGET = "ClassLabel"

df = df.drop_duplicates().copy()
df[TARGET] = pd.to_numeric(df[TARGET], errors="coerce")
df = df[df[TARGET].isin([0, 1]).copy()]
df[TARGET] = df[TARGET].astype(int)

available_features = [c for c in FEATURES if c in df.columns]
for c in available_features:
    df[c] = pd.to_numeric(df[c], errors="coerce")

X = df[available_features]
y = df[TARGET]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
)

print("Features used:", len(available_features))
print("Training rows:", len(X_train))
print("Testing rows:", len(X_test))

```

```

Features used: 16
Training rows: 80697
Testing rows: 20175

```

✓ 6. Evaluation helper

```

results = []

def phishing_probability(model, X):
    if hasattr(model, "predict_proba"):
        probs = model.predict_proba(X)
        classes = list(model.classes_)
        return probs[:, classes.index(0)]
    if hasattr(model, "decision_function"):
        return -np.asarray(model.decision_function(X))
    return None

def evaluate(name, y_true, y_pred, score=None):
    row = {
        "Model": name,
        "Accuracy": accuracy_score(y_true, y_pred),

```

```

        "Precision_Phishing": precision_score(y_true, y_pred, pos_label=0, zero_division=0),
        "Recall_Phishing": recall_score(y_true, y_pred, pos_label=0, zero_division=0),
        "F1_Phishing": f1_score(y_true, y_pred, pos_label=0, zero_division=0),
        "ROC_AUC": np.nan
    }

    if score is not None:
        row["ROC_AUC"] = roc_auc_score((np.asarray(y_true) == 0).astype(int), score)

    results.append(row)

    print("\n", "="*65)
    print(name)
    print("="*65)
    print(classification_report(
        y_true, y_pred,
        labels=[0,1],
        target_names=["Phishing (0)", "Legitimate (1)"],
        digits=4
    ))

    cm = confusion_matrix(y_true, y_pred, labels=[0,1])
    disp = ConfusionMatrixDisplay(cm, display_labels=["Phishing", "Legitimate"])
    disp.plot(values_format="d")
    plt.title(name + " - Confusion Matrix")
    plt.show()

    if score is not None:
        RocCurveDisplay.from_predictions(
            (np.asarray(y_true) == 0).astype(int), score
        )
        plt.title(name + " - ROC Curve")
        plt.show()

    return row

```

✓ 7. Model 1 — Logistic Regression

```

lr = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=2000, class_weight="balanced", random_state=42))
])
lr.fit(X_train, y_train)
pred = lr.predict(X_test)
evaluate("Logistic Regression", y_test, pred, phishing_probability(lr, X_test))

```



```
\n =====
Logistic Regression
=====
```

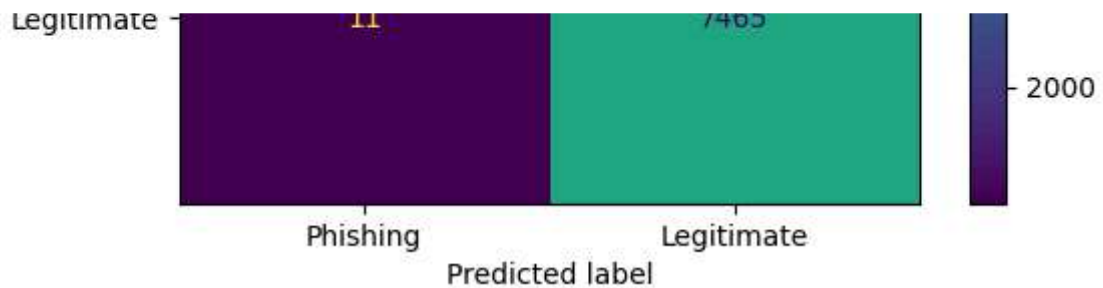
	precision	recall	f1-score	support
Phishing (0)	0.9991	0.9961	0.9976	12699
Legitimate (1)	0.9935	0.9985	0.9960	7476
accuracy			0.9970	20175
macro avg	0.9963	0.9973	0.9968	20175
weighted avg	0.9970	0.9970	0.9970	20175

Logistic Regression - Confusion Matrix

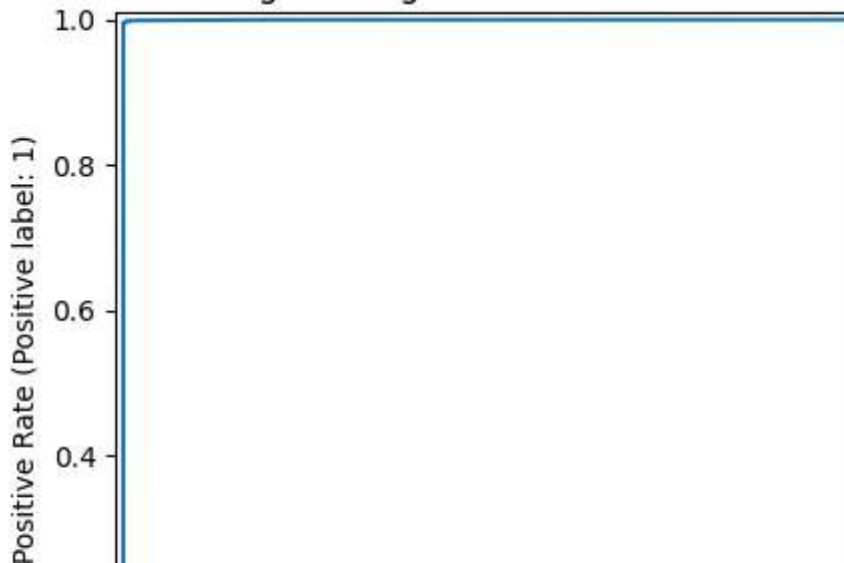


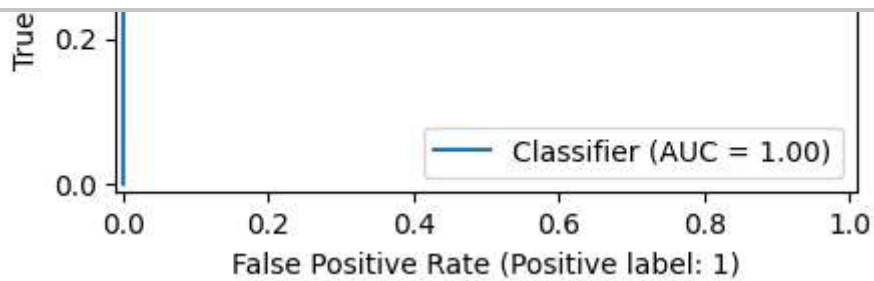
8. Model 2 — Support Vector Machine (SVM)

```
svm = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("model", SVC(kernel="rbf", probability=True, class_weight="balanced", ranc
])
svm.fit(X_train, y_train)
pred = svm.predict(X_test)
evaluate("SVM", y_test, pred, phishing_probability(svm, X_test))
```



Logistic Regression - ROC Curve





```
{'Model': 'Logistic Regression',  
 'Accuracy': 0.9970260223048327,  
 'Precision_Phishing': 0.9991311902693311,  
 'Recall_Phishing': 0.9961414284589337,  
 'F1_Phishing': 0.9976340694006309,  
 'ROC_AUC': np.float64(0.9998049668854501)}
```

```
\n =====
SVM
=====
```

	precision	recall	f1-score	support
Phishing (0)	0.9998	0.9980	0.9989	12699
Legitimate (1)	0.9965	0.9996	0.9981	7476
accuracy			0.9986	20175
macro avg	0.9981	0.9988	0.9985	20175
weighted avg	0.9986	0.9986	0.9986	20175

SVM - Confusion Matrix

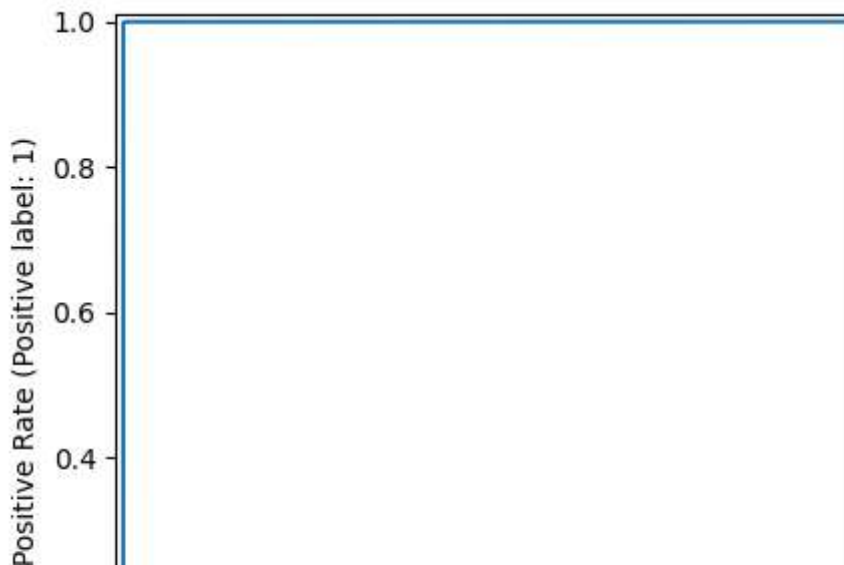


9. Model 3 — Random Forest

```
rf = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("model", RandomForestClassifier(
        n_estimators=300, class_weight="balanced",
        random_state=RANDOM_STATE, n_jobs=-1
    ))
])
rf.fit(X_train, y_train)
pred = rf.predict(X_test)
evaluate("Random Forest", y_test, pred, phishing_probability(rf, X_test))
```



SVM - ROC Curve





```
{'Model': 'SVM',  
 'Accuracy': 0.9985625774473358,  
 'Precision_Phishing': 0.9997633322814768,  
 'Recall_Phishing': 0.9979525946924954,  
 'F1_Phishing': 0.9988571428571429,  
 'ROC_AUC': np.float64(0.9999213168413433)}
```

```
\n =====
Random Forest
=====
```

	precision	recall	f1-score	support
Phishing (0)	0.9993	0.9999	0.9996	12699
Legitimate (1)	0.9999	0.9988	0.9993	7476
accuracy			0.9995	20175
macro avg	0.9996	0.9994	0.9995	20175
weighted avg	0.9995	0.9995	0.9995	20175

Random Forest - Confusion Matrix



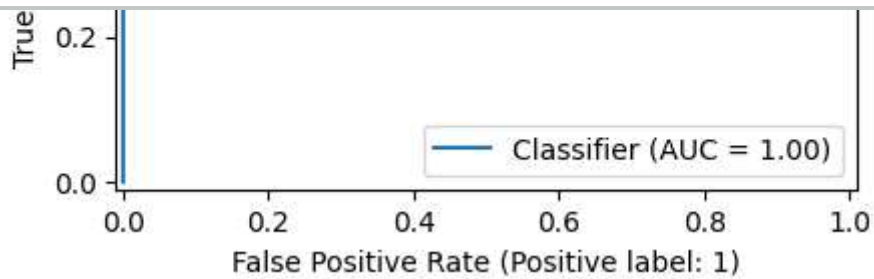
10. Model 4 — XGBoost

```
xgb = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("model", XGBClassifier(
        n_estimators=300, max_depth=6, learning_rate=0.08,
        subsample=0.9, colsample_bytree=0.9,
        objective="binary:logistic", eval_metric="logloss",
        random_state=RANDOM_STATE, n_jobs=-1
    ))
])
xgb.fit(X_train, y_train)
pred = xgb.predict(X_test)
evaluate("XGBoost", y_test, pred, phishing_probability(xgb, X_test))
```



Random Forest - ROC Curve



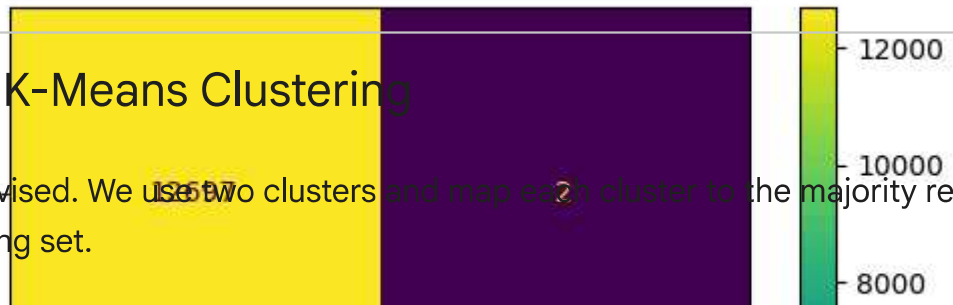


```
{'Model': 'Random Forest',  
 'Accuracy': 0.9995043370508054,  
 'Precision_Phishing': 0.9992917289682852,  
 'Recall_Phishing': 0.999921253642019,  
 'F1_Phishing': 0.999606392190821,  
 'ROC_AUC': np.float64(0.9999983252179081)}
```

```
\n =====
XGBoost
=====
```

	precision	recall	f1-score	support
Phishing (0)	0.9995	0.9998	0.9997	12699
Legitimate (1)	0.9997	0.9992	0.9995	7476
accuracy			0.9996	20175
macro avg	0.9996	0.9995	0.9996	20175
weighted avg	0.9996	0.9996	0.9996	20175

XGBoost - Confusion Matrix



11. Model 5 — K-Means Clustering

K-Means is unsupervised. We use two clusters and map each cluster to the majority real class from the training set.

```
kmeans_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("model", KMeans(n_clusters=2, n_init=20, random_state=RANDOM_STATE))
])

train_clusters = kmeans_pipe.fit_predict(X_train)
cluster_map = {}

y_train_np = y_train.to_numpy()
for c in [0, 1]:
    labels = y_train_np[train_clusters == c]
    counts = np.bincount(labels.astype(int), minlength=2)
    cluster_map[c] = int(np.argmax(counts))

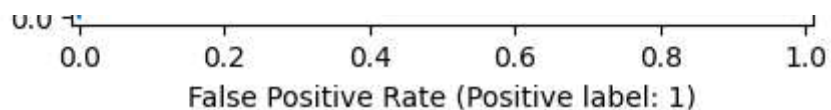
test_clusters = kmeans_pipe.predict(X_test)
kmeans_pred = np.array([cluster_map[int(c)] for c in test_clusters])

Xt = kmeans_pipe[:-1].transform(X_test)
dist = kmeans_pipe.named_steps["model"].transform(Xt)

phish_clusters = [c for c, label in cluster_map.items() if label == 0]
legit_clusters = [c for c, label in cluster_map.items() if label == 1]

if phish_clusters and legit_clusters:
    dp = dist[:, phish_clusters[0]]
    dl = dist[:, legit_clusters[0]]
    kmeans_score = dl / (dp + dl + 1e-12)
else:
    kmeans_score = (kmeans_pred == 0).astype(float)
```

```
print("Cluster-to-class mapping:", cluster_map)
evaluate("K-Means Clustering", y_test, kmeans_pred, kmeans_score)
```



```
{'Model': 'XGBoost',
 'Accuracy': 0.9996034696406444,
 'Precision_Phishing': 0.9995276706289853,
 'Recall_Phishing': 0.9998425072840381,
 'F1_Phishing': 0.9996850641681757,
 'ROC_AUC': np.float64(0.9999984094836736)}
```


Cluster-to-class mapping: {0: 1, 1: 0}

\n =====

K-Means Clustering

=====

	precision	recall	f1-score	support
Phishing (0)	0.9996	0.8060	0.8925	12699
Legitimate (1)	0.7521	0.9995	0.8583	7476
accuracy			0.8777	20175
macro avg	0.8758	0.9028	0.8754	20175
weighted avg	0.9079	0.8777	0.8798	20175

K-Means Clustering - Confusion Matrix



12. Compare the first five models

```
comparison_5 = pd.DataFrame(results).sort_values(
    ["Recall_Phishing", "F1_Phishing", "ROC_AUC"],
    ascending=False
).reset_index(drop=True)
```

```
display(comparison_5.style.format({
    "Accuracy": "{:.4f}",
    "Precision_Phishing": "{:.4f}",
    "Recall_Phishing": "{:.4f}",
    "F1_Phishing": "{:.4f}",
    "ROC_AUC": "{:.4f}",
}))
```

	Model	Accuracy	Precision_Phishing	Recall_Phishing	F1_Phishing	ROC_AUC
0	Random Forest	0.9995	0.9993	0.9999	0.9996	1.0000
1	XGBoost	0.9996	0.9995	0.9998	0.9997	1.0000
2	SVM	0.9986	0.9998	0.9980	0.9989	0.9999
3	Logistic Regression	0.9970	0.9991	0.9961	0.9976	0.9999

13. Model 6 — Large Language Model (LLM)

For tabular data, every row is converted into a text description such as:

url_length=120, has_ip_address=1, url_entropy=4.8, ...

The instruction LLM then returns either `phishing` or `legitimate`.

The default is `Qwen/Qwen2.5-0.5B-Instruct`.

To keep Colab runtime practical, evaluate a subset such as 100–200 test rows.

Classifier (AUC = 0.99)

```
!pip uninstall -y sympy transformers tokenizers
!pip install -q sympy==1.13.1
!pip install -q transformers==4.46.3 tokenizers==0.20.3 accelerate
```

```
{
  'Accuracy': 0.8777199504337051,
  'Found-existing-installation': transformers 4.46.3,
  'Uninstall-Phishing': 1018060477202929365,
  'E1-Phishing': 0.8924538994725141,
  'Successfully-uninstalled-sympy': 1.13.3,
  'ROC_AUC': np.float64(0.9881619765816168)}
Found-existing installation: transformers 4.46.3
Uninstalling transformers-4.46.3:
```

```
  Successfully uninstalled transformers-4.46.3
```

```
Found existing installation: tokenizers 0.20.3
```

```
Uninstalling tokenizers-0.20.3:
```

```
  Successfully uninstalled tokenizers-0.20.3
```

```
ERROR: pip's dependency resolver does not currently take into account all the pa
peft 0.20.0 requires transformers, which is not installed.
```

```
sentence-transformers 5.7.0 requires tokenizers>=0.19, which is not installed.
```

```
sentence-transformers 5.7.0 requires transformers<6.0.0,>=4.41.0, which is not i
torch 2.11.0+cu128 requires sympy>=1.13.3, but you have sympy 1.13.1 which is in
```

```
!pip install -q --upgrade sympy==1.13.3
```

```
import torch
import sympy
import transformers
```

```
print("Torch:", torch.__version__)
print("SymPy:", sympy.__version__)
print("Transformers:", transformers.__version__)
print("GPU available:", torch.cuda.is_available())
```

```
from transformers import AutoTokenizer, AutoModelForCausalLM
print("Everything is ready!")
```

```
Torch: 2.11.0+cu128
SymPy: 1.13.3
Transformers: 4.46.3
GPU available: True
Everything is ready!
```

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
```

```

LLM_MODEL = "Qwen/Qwen2.5-0.5B-Instruct"
LLM_SAMPLES = 200 # Increase to 200 or more if you have enough runtime.

tokenizer = AutoTokenizer.from_pretrained(LLM_MODEL)

model_kwargs = {}
if torch.cuda.is_available():
    model_kwargs["torch_dtype"] = torch.float16
    model_kwargs["device_map"] = "auto"

llm = AutoModelForCausalLM.from_pretrained(LLM_MODEL, **model_kwargs)
if not torch.cuda.is_available():
    llm = llm.to("cpu")

test_idx = X_test.sample(
    n=min(LLM_SAMPLES, len(X_test)),
    random_state=RANDOM_STATE
).index

X_llm = X_test.loc[test_idx]
y_llm = y_test.loc[test_idx]

def tabular_row_to_prompt(row):
    feature_text = ", ".join(
        f"{col}={row[col]}" for col in available_features
    )
    return (
        "You are a cybersecurity classifier. "
        "Classify this URL using only the following tabular lexical and structu
        "Reply with exactly one word: phishing or legitimate.\\n"
        f"Features: {feature_text}\\n"
        "Answer:"
    )

def parse_llm_answer(text):
    text = text.lower()
    if "phishing" in text:
        return 0
    if "legitimate" in text or "benign" in text or "safe" in text:
        return 1
    return 0

llm_preds = []
llm_raw = []

for i, (_, row) in enumerate(X_llm.iterrows(), 1):
    prompt = tabular_row_to_prompt(row)
    messages = [{"role": "user", "content": prompt}]

    if tokenizer.chat_template:

```

```
        rendered = tokenizer.apply_chat_template(
            messages, tokenize=False, add_generation_prompt=True
        )
    else:
        rendered = prompt

    inputs = tokenizer(rendered, return_tensors="pt")
    inputs = {k: v.to(llm.device) for k, v in inputs.items()}

    with torch.no_grad():
        output = llm.generate(
            **inputs,
            max_new_tokens=8,
            do_sample=False,
            pad_token_id=tokenizer.eos_token_id
        )

    generated = output[0][inputs["input_ids"].shape[1]:]
    answer = tokenizer.decode(generated, skip_special_tokens=True).strip()

    llm_raw.append(answer)
    llm_preds.append(parse_llm_answer(answer))

    if i % 20 == 0 or i == len(X_llm):
        print(f"LLM processed {i}/{len(X_llm)}")

evaluate("LLM", y_llm.to_numpy(), np.array(llm_preds), score=None)
for i in range(10):
    print("True:", y_llm.iloc[i])
    print("LLM output:", llm_raw[i])
    print("Predicted:", llm_preds[i])
    print("---")
```